Transport Layer

TRANSPORT LAYER

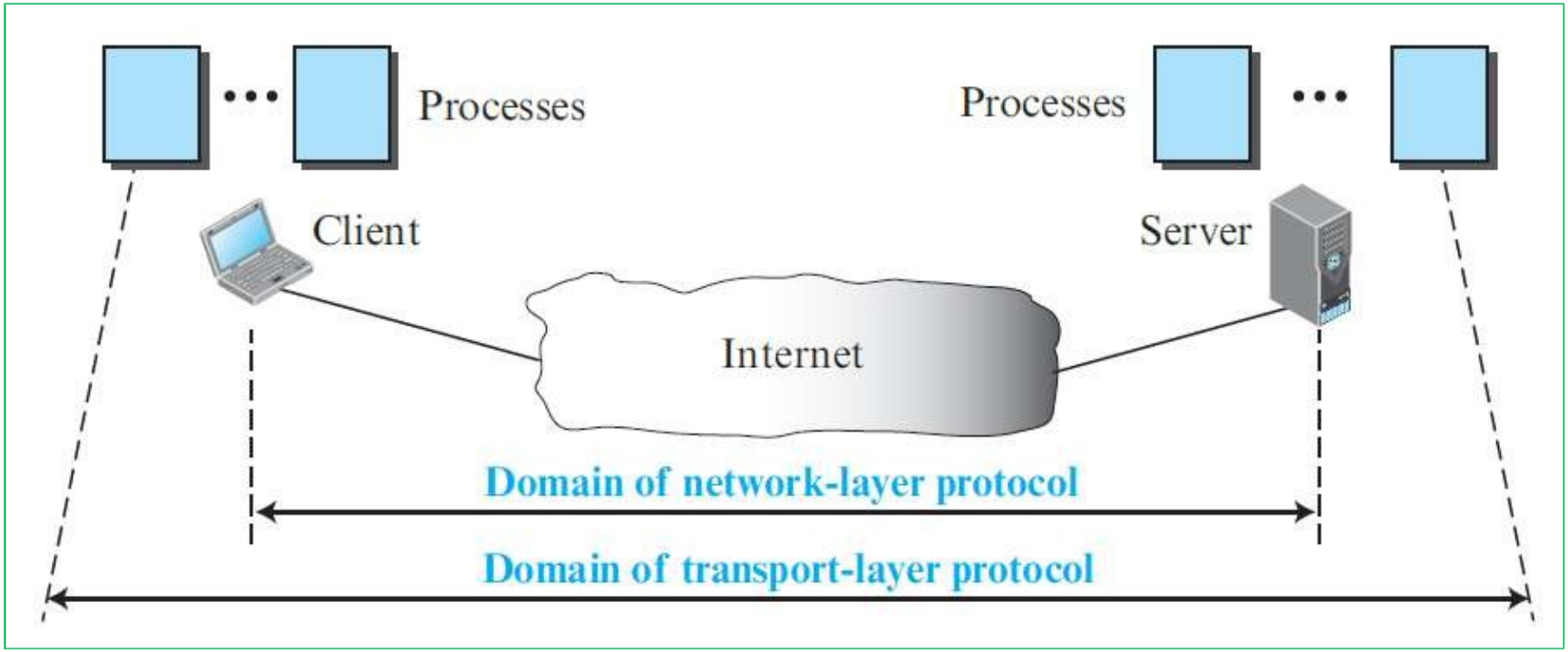
- The transport layer is responsible for the delivery of a message from one process to another
- A process is an application program running on a host
- The transport layer header must include a type of address called a service-point address in the OSI model and port number or port addresses in the Internet and TCP/IP protocol suite.
- A transport layer protocol can be either connectionless or connection-oriented.
- In the transport layer, a message is normally divided into transmittable segments

TRANSPORT LAYER SERVICES

❖Process-to-Process Communication

- The first duty of a transport-layer protocol is to provide process-to- process communication.
- A process is an application-layer entity (running program) that uses the services of the transport layer.
- A network-layer protocol can deliver the message only to the destination computer. However, this is an incomplete delivery. The message still needs to be handed to the correct process. This is where a transport – layer protocol takes over. A transport-layer protocol is responsible for delivery of the message to the appropriate process.

Network layer versus transport layer

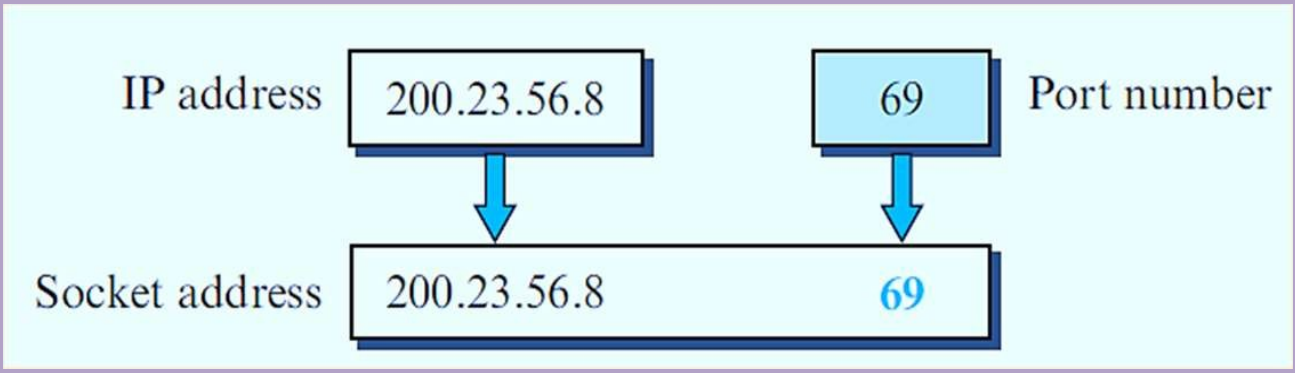


❖Addressing: Port Numbers

- For communication, we must define the local host, local process, remote host, and remote process.
- The local host and the remote host are defined using IP addresses To define the processes, we need second identifiers, called port numbers.
- In the TCP/IP protocol suite, the port numbers are integers between 0 and 65,535 (16 bits).
- TCP/IP has decided to use universal port numbers for servers; these are called **well-known port numbers**.
- Every client process knows the well-known port number of the corresponding server process

Socket Addresses

- A transport-layer protocol in the TCP suite needs both the IP address and the port number, at each end, to make a connection. The combination of an IP address and a port number is called a socket address. The client socket address defines the client process uniquely just as the server socket address defines the server process uniquely.



- **The main protocols of Transport Layer:**

    1. TCP(Transmission Control Protocol).
    2. UDP(User Datagram Protocol).

- **Services Provided by Transport Layer:**

    1. Connection-Oriented Service.
    2. Connectionless service.

- **Segmentation and Reassembly:** This layer accepts the message from the (session) layer, and breaks the message into smaller units. Each of the segments produced has a header associated with it. The transport layer at the destination station reassembles the message.

- **Error Control** : **checksums** guarantee that the data transmitted is the same as the data received and that is not corrupt.
- **Flow control** : It ensures that the data is sent at a rate that is acceptable for both sides by managing data flow.
- **Congestion Control** : This network congestion can affect almost every part of a network. The transport layer can identify the symptoms of overloaded nodes and reduced flow rates and take the proper steps to remediate these issues.

❖ Multiplexing and Demultiplexing

- Whenever an entity accepts items from more than one source, this is referred to as multiplexing (many to one).
- whenever an entity delivers items to more than one source, this is referred to as demultiplexing (one to many).
- The transport layer at the source performs multiplexing; the transport layer at the destination performs demultiplexing.

Sequence Numbers

- Error control requires that the sending transport layer knows which packet is to be resent and the receiving transport layer knows which packet is a duplicate, or which packet has arrived out of order.
- This can be done if the packets are numbered. We can add a field to the transport-layer packet to hold the sequence number of the packet.
- When a packet is corrupted or lost, the receiving transport layer can somehow inform the sending transport layer to resend that packet using the sequence number.
- The receiving transport layer can also detect duplicate packets if two received packets have the same sequence number.

- The out-of-order packets can be recognized by observing gaps in the sequence numbers.

➢ **Acknowledgment**

- We can use both positive and negative signals as error control.
- positive signals are more common at the transport layer.
- The receiver side can send an acknowledgment (ACK) for each of a collection of packets that have arrived safe and sound.
- The receiver can simply discard the corrupted packets. The sender can detect lost packets if it uses a timer. When a packet is sent, the sender starts a timer. If an ACK does not arrive before the timer expires, the sender resends the packet.

- Duplicate packets can be silently discarded by the receiver. Out-of- order packets can be either discarded (to be treated as lost packets by the sender), or stored until the missing ones arrives.

❖ Connectionless and Connection-Oriented Services

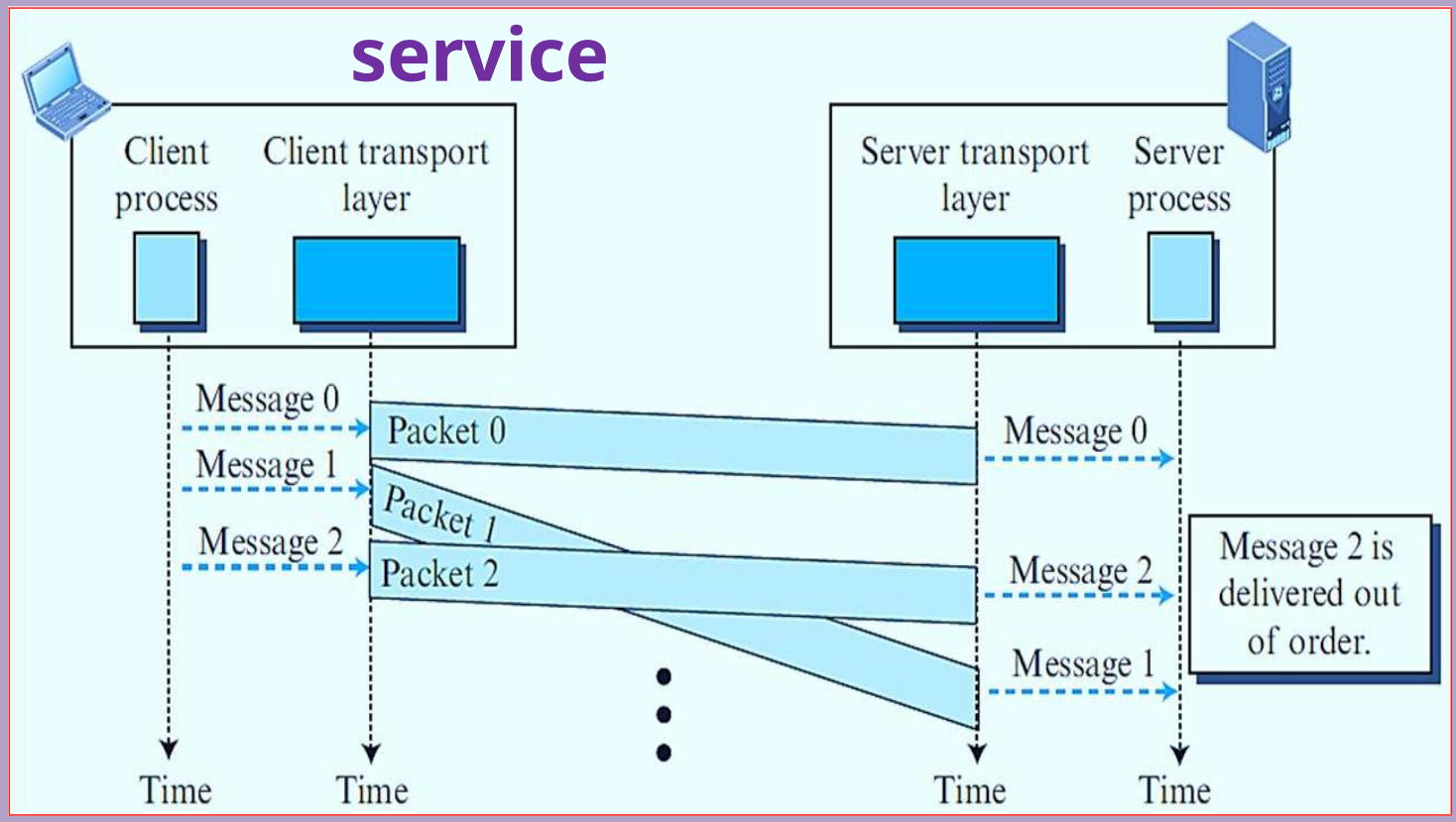
- In a connectionless service, the source process (application program) needs to divide its message into chunks of data of the size acceptable by the transport layer and deliver them to the transport layer one by one.
- The transport layer treats each chunk as a single unit without any relation between the chunks.
- When a chunk arrives from the application layer, the transport layer encapsulates it in a packet and sends it.

- However, since there is no dependency between the packets at the transport layer, the packets may arrive out of order at the destination and will be delivered out of order to the server process.
- The situation would be worse if one of the packets were lost. Since there is no numbering on the packets, the receiving transport layer has no idea that one of the messages has been lost.
- We can say that no flow control, error control, or congestion control can be effectively implemented in a connectionless service.

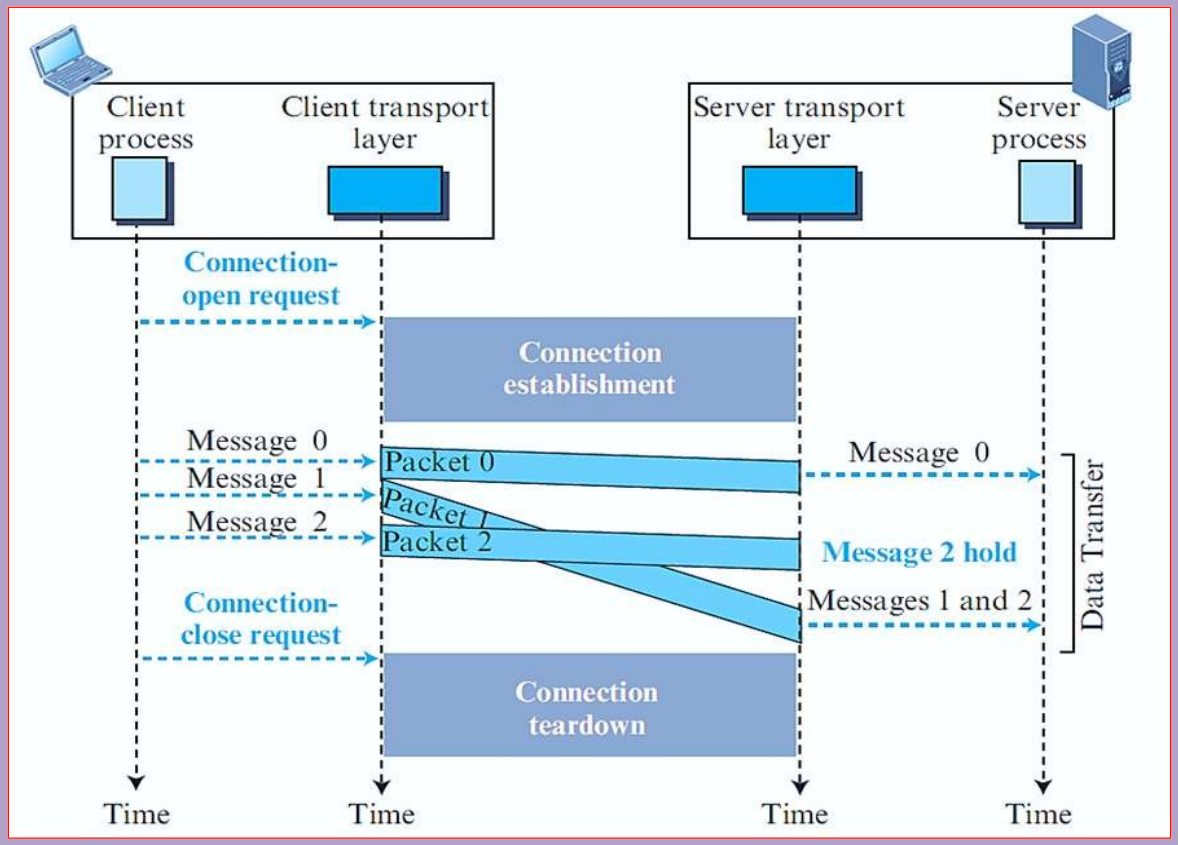
Connection-Oriented Service

- In a connection-oriented service, the client and the server first need to establish a logical connection between themselves.
- The data exchange can only happen after the connection establishment.
- After data exchange, the connection needs to be torn down.

Connectionless service



Connection Oriented



TCP CONNECTION

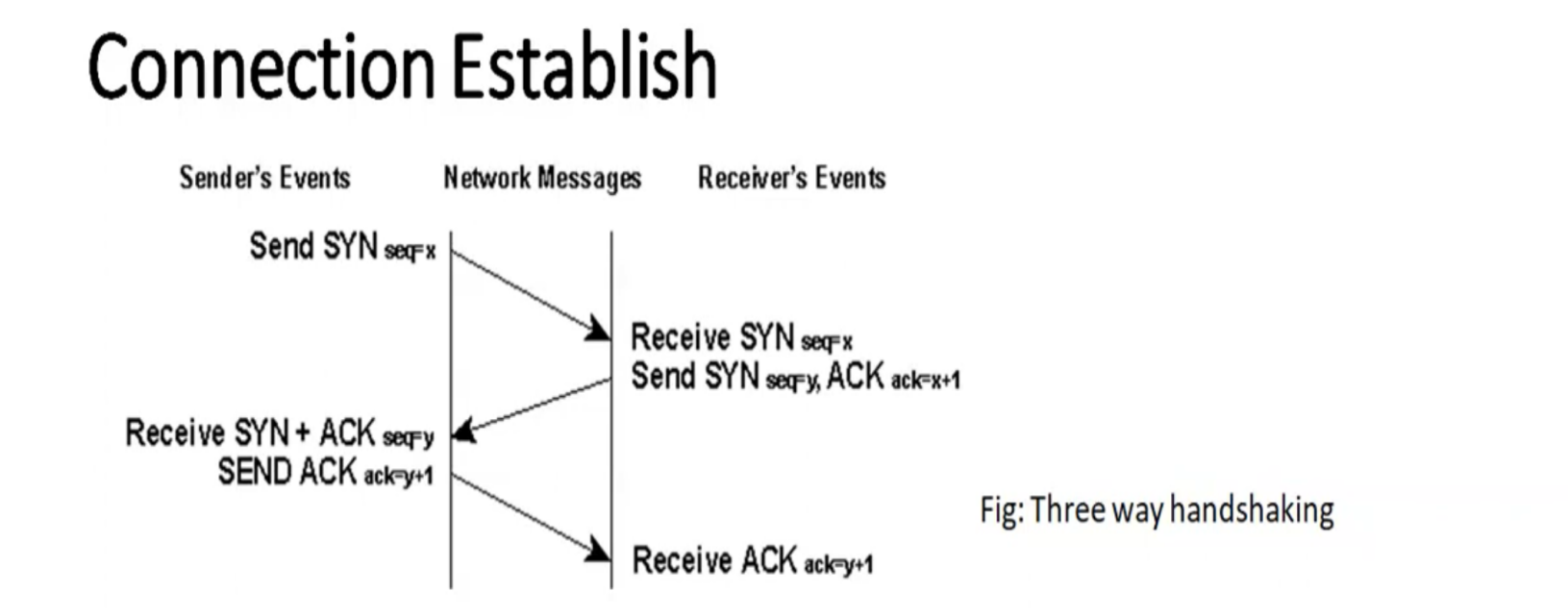
- In TCP, connection-oriented transmission requires three phases: connection establishment, data transfer, and connection termination

❖ Connection Establishment

- TCP transmits data in full-duplex mode. When two TCPs in two machines are connected, they are able to send segments to each other simultaneously

➢ Three-Way Handshaking

- The connection establishment in TCP is called three-way handshaking



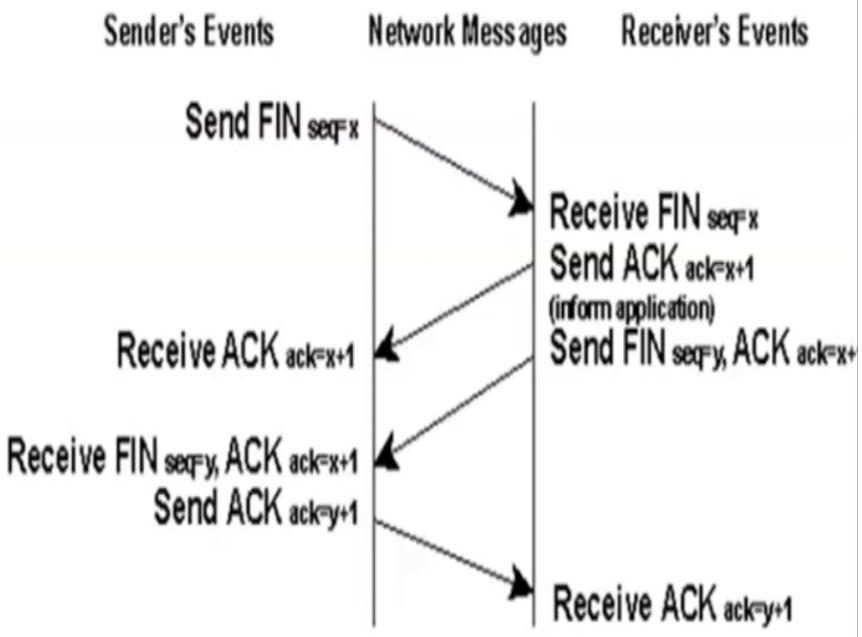
- The sender sends a SYN packet with sequence number say ‘X’.
- The receiver on receiving SYN packet responds with SYN packet with sequence number 'y' and ACK with seq number ‘X+ 1'
- On receiving both SYN and ACK packet, the sender

Data Transfer

- After connection is established, bidirectional data transfer can take place.
- The **client and server** can both send data and acknowledgements.

Connection Release/Termination

- The **initiator** sends a **FIN** with the current sequence and acknowledgement number.
- The **responder** on receiving this informs the application program
  - that it will receive no more data and
  - sends an acknowledgement of the packet.
- The **connection** is now closed from one side.
- Now the **responder** will follow similar steps to close the connection from its side.
- Once this is done the **connection** will be fully closed.

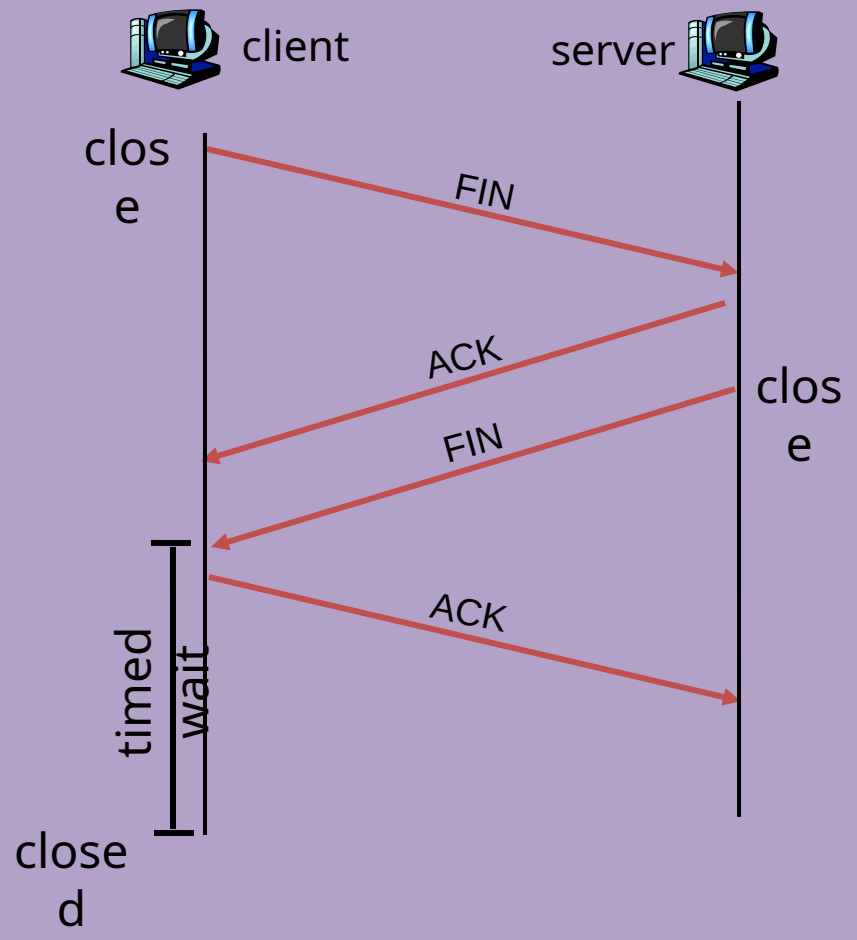


TCP Connection Management (cont.)

Closing a connection:

Step 1: client end system sends TCP FIN control segment to server

Step 2: server receives FIN, replies with ACK. Closes connection, sends FIN.



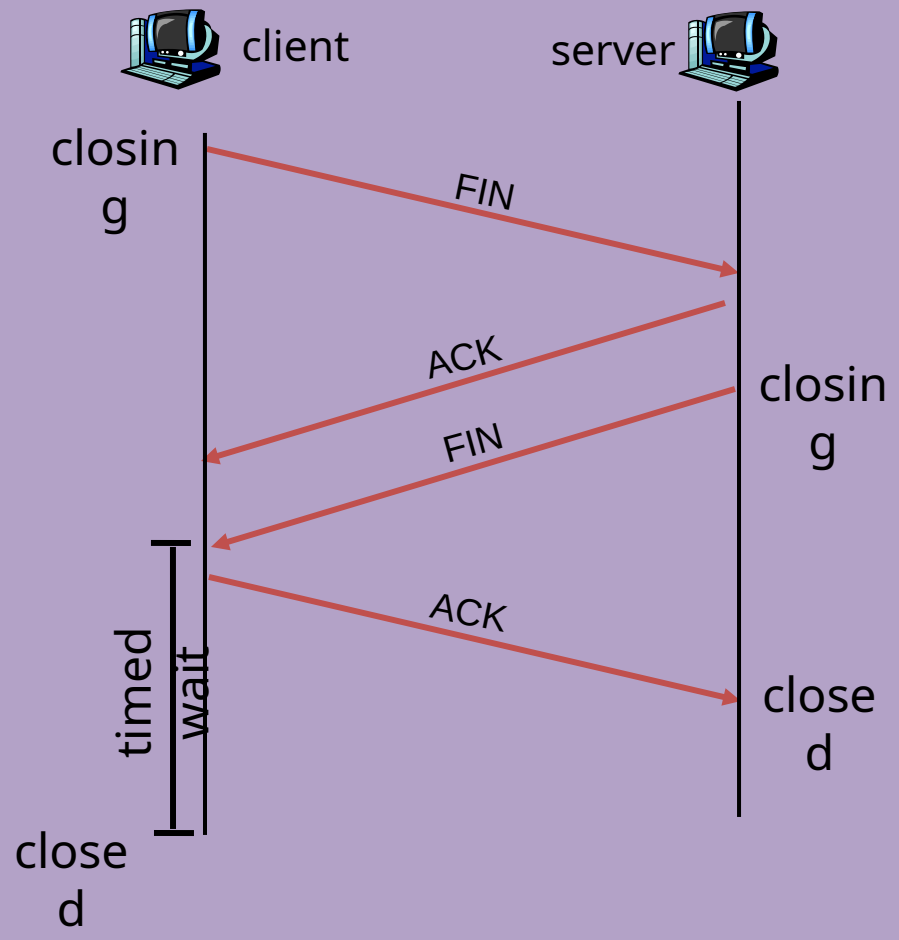
TCP Connection Management (cont.)

Step 3: client receives FIN, replies with ACK.

- Enters “timed wait” - will respond with ACK to received FINs

Step 4: server, receives ACK. Connection closed.

Note: with small modification, can handle simultaneous FINs.



TCP

Three-way handshaking:

- The connection establishment in TCP is called **three way handshaking**.
  - The process starts with the **server**.
  - The **server** program
    - **tells its TCP** that it is ready to accept a connection.
    - This is called a **request** for a **passive open**.
  - The **client** program
    - issues a **request** for an active open.
  - A **client** that wishes to connect to an open server
    - **tells its TCP** that it needs to be connected to that particular **server**.
    - **TCP** can now start the **three-way handshaking** process.

TCP Vs UDP

TCP	UDP
➢ It is a connection-oriented protocol.	➢ It is a connectionless protocol.
➢ TCP is a reliable protocol as it provides assurance for the delivery of data packets.	➢ UDP is an unreliable protocol as it does not take the guarantee for the delivery of packets.
➢ TCP is slower than UDP as it performs error checking, flow control, and provides assurance for the delivery of data packets.	➢ UDP is faster than TCP as it does not guarantee the delivery of data packets.
➢ TCP has a (20-60) bytes variable length header	➢ UDP has an 8 bytes fixed-length header.
➢ TCP performs flow control and congestion control mechanisms.	➢ This protocol follows no such mechanisms.
➢ Error control is mandatory.	➢ Error control is optional.

➢ **Overhead is higher than UDP.**	➢ **Overhead is low**
➢ TCP is used by HTTP, FTP, SMTP ,TELNET etc.	➢ UDP is used by DNS, DHCP, RIP etc.
➢ TCP doesn't support Broadcasting.	➢ UDP supports Broadcasting.